PR 2: `exact` scheme specification for MOI

Review document, 10 September 2026. Prepared for internal review before filing.

Page 1: what this PR is

Target repository: `x402-foundation/x402` (public). One new file, `specs/schemes/exact/scheme_exact_moi.md`. No code.

PR title: specs: add `exact` scheme for MOI (`moi:*`), upfront flow

Why it exists: the x402 repo lands a scheme specification before it reviews a mechanism package. This document is the contract PR 3 implements, and it is what a reviewer reads to decide whether MOI's flow is sound.

High-level requirements

The file follows the structure of the merged `stellar`, `near` and `ton` specs. Each section below is required.

Section	What it must contain
Versions supported	Which x402 versions the scheme works with, and why not the others
Supported Networks	CAIP-2 identifiers a facilitator can settle on, with a scope note
Summary	The payment flow (`upfront`, `authorization` or `escrow`), why, and its consequences
Protocol Flow	Numbered steps from first request to `PAYMENT-RESPONSE`
`PaymentRequirements` for `exact`	The JSON the seller emits, with every field defined
`PAYMENT-SIGNATURE` Header Payload	The JSON the buyer sends, the canonical bytes it signs, and why
Facilitator Settlement Rules (MUST)	Numbered rules a facilitator must enforce, each testable
`PAYMENT-RESPONSE` Header Payload	The JSON returned to the buyer on success and failure
Duplicate Settlement Mitigation	The known race and the recommended fix
Security Considerations	What the scheme accepts, rejects and assumes

Section	What it must contain
Appendix	Chain facts a reviewer needs and cannot look up elsewhere

Status

- Complete. Every section walked and compared with the merged specs; identifier layout, RPC field names, escaping rule, `network` in the claim, `areFeesSponsored` and the fee-payer note were all corrected during review.
- Every protocol claim was verified against go-moi `develop` (2026-09-07) and the live Voyage devnet; the package in PR 3 passes 36 unit tests against this spec.
- **Waits on PR 1.** The spec names `moi:devnet`; the PR must not merge before the CASA namespace does. If the reference format becomes numeric, Supported Networks and every example change with it.
- One line to re-check on the wire once moipod v0.13.0 is live: the appendix says the fee payer field is `fee_payer` (it was `payer`).

Index

1. PR description (as it will be posted)
2. File: `specs/schemes/exact/scheme_exact_moi.md`, with a context note under every section
    - 2.1 Versions supported
    - 2.2 Supported Networks
    - 2.3 Summary
    - 2.4 Protocol Flow
    - 2.5 `PaymentRequirements` for `exact`
    - 2.6 `PAYMENT-SIGNATURE` Header Payload, canonical claim bytes, why a claim exists
    - 2.7 Facilitator Settlement Rules 1 to 8
    - 2.8 `PAYMENT-RESPONSE` Header Payload
    - 2.9 Duplicate Settlement Mitigation
    - 2.10 Security Considerations
    - 2.11 Appendix: MOI in brief, interaction and receipt anatomy, POLO, references

1. PR description

PR description — x402-foundation/x402 (spec only)

Title: specs: add `exact` scheme for MOI (`moi:*`), upfront flow

Adds `specs/schemes/exact/scheme_exact_moi.md`. Specification only, no code — per the repo's process, the spec lands before any mechanism package is reviewed.

This scheme uses the `upfront` payment flow (spec v2 section 6.1): the buyer settles a protocol-native asset transfer first and proves it with a signed claim bound to one resource. Per the upfront ordering, `/verify` is omitted and all validation runs inside `/settle`, which confirms the transfer by reading the interaction and its receipt back from the chain. The facilitator holds no keys and sponsors no fuel, making self-facilitation the natural deployment.

`upfront` is a choice, not a limit. MOI supports the other two flows natively — `escrow` through the asset standard's `Lockup`/`Release` endpoints or a Coco escrow logic, and `authorization` through third-party relay of a signed interaction with a co-signing fee payer (moipod v0.13.0) or the `Approve`/`TransferFrom` allowance. Those are intended as follow-up scheme specs once this one merges; this PR keeps to one flow so the settlement rules stay small enough to verify against a live network.

The spec follows the structure of the merged stellar/near/ton specs: versions, supported networks, protocol flow, MUST-rule settlement checks, PAYMENT-RESPONSE, and duplicate-settlement mitigation.

Namespace status: the `moi` CAIP-2 namespace is under review at ChainAgnostic/namespaces#TODO. This PR should not merge before that one does.

- ☐ CAIP-2 namespace PR to CASA opened (spec names `moi:*` networks)
- ☐ Commits GPG-signed, conventional prefixes
- ☐ AI assistance disclosed (this PR was drafted with AI assistance and human-reviewed)
- ☐ Internal review (Rahul) complete

2. File: `specs/schemes/exact/scheme_exact_moi.md`

Scheme: `exact` on `MOI`

> **Context.** One file, `specs/schemes/exact/scheme_exact_moi.md`. It sits beside the merged `scheme_exact_stellar.md`, `scheme_exact_near.md` and `scheme_exact_ton.md` and follows their section order so a maintainer can diff it against a known shape.

Versions supported

> **Context.** x402 v1 carried a closed enum of network names, so no MOI network can be expressed in it; every merged non-EVM spec marks v1 unsupported the same way. v2 names networks by CAIP-2 string, which is why PR 1 has to exist before this one.

- ❌ `v1` — not supported; v1's closed network enum cannot name a MOI network.
- ✅ v2

Supported Networks

> **Context.** Only the live public network. `moi:mainnet` is reserved in the CASA doc but deliberately not listed here, because a scheme spec lists what a facilitator can settle on today. The scope note carries the two facts every later rule depends on: native fungible assets only, and no fuel sponsorship by the facilitator.

- `moi:devnet` — Voyage devnet (CASA namespace)

> [!NOTE] **Scope:** This spec covers transfers of protocol-native fungible assets only (MAS0, and MASN for KMOI). The facilitator does not sponsor fuel.

Summary

> **Context.** The most-read section. It names the flow (`upfront`, x402 v2 section 6.1) and frames it as a choice, listing the two other flows so a reviewer does not read `upfront` as a limitation. The escrow bullet was reworded on 10 September after the source check showed a Coco logic can hold and release funds by code. The two consequence bullets are exactly what PR 3 implements: no `/verify`, every rule in `/settle`, a keyless facilitator.

The buyer pays exactly the quoted amount in a protocol-native MOI asset (MAS0, or MASN for KMOI).

MOI uses the `upfront` payment flow (x402 v2, section 6.1): the buyer settles the transfer on chain first, then presents a signed claim naming that settled transfer as proof.

`upfront` is a choice. MOI can express the other two flows; this scheme does not use them:

- `authorization`. A signed MOI interaction can be relayed by a third party through `moi.SendInteractions`, and from moipod v0.13.0 a separate fee payer can co-sign and pay the fuel. This scheme does not use that path because a per-request payment should not leave a signed, submittable instruction in a third party's hands between

request and settlement. The native `Approve`/`TransferFrom` allowance is the other route; it costs the buyer an extra on-chain interaction per grant.

- `escrow`. MOI can hold funds pending release, either natively (`Lockup`/`Release`, where a named releaser account moves the funds) or in a Coco logic that holds the deposit in its own account and releases it by code. Either way a second interaction sits between payment and finality. A first scheme should settle in one.

`upfront` uses one interaction, is final on commit, and leaves nothing pending on either side.

Consequences:

- Per the `upfront` ordering (settle → resource → respond), the facilitator's read-only `/verify` endpoint is not part of the flow. All validation defined below MUST be enforced within `/settle`, which confirms the already-submitted transfer rather than executing one.
- The facilitator holds no keys and signs nothing (`getSigners()` returns an empty list), which makes in-process self-facilitation the natural deployment. Fuel is the buyer's, or a fee payer's the buyer names; the scheme does not distinguish.

Protocol Flow

Context. Eight numbered steps, the shape the merged specs use. Steps 3 and 4 are the MOI-specific part: one `ASSET_INVOKE` operation calling the asset's `Transfer` callsite, then a wait for the receipt. Step 6 states the omission of `/verify`, which follows from core's upfront ordering (settle, resource, respond). The RPC names in step 7 are the exact methods the package's chain reader calls.

1. **Client** requests a protected resource.
2. **Resource Server** responds with the payment required signal; the `PaymentRequired` object travels in the `PAYMENT-REQUIRED` header. `accepts[].extra` carries `"paymentFlow": "upfront"`, which core emits automatically because the resolved flow is not `authorization`, and `"resource"`, which the scheme's server half copies from the request's `ResourceInfo` so the buyer can sign over it.
3. **Client** submits an interaction on the named MOI network carrying one `ASSET_INVOKE` operation that calls `Transfer` on the quoted asset: the quoted amount to `payTo`, signed with its own key. Fuel comes from the client or from a fee payer it names.
4. **Client** waits until `moi.InteractionReceipt` returns a successful receipt for the interaction, then constructs and signs a payment claim (below) binding that settled transfer to this request.
5. **Client** repeats the request with the claim in the `PAYMENT-SIGNATURE` header.
6. **Resource Server** passes the payload and requirements to the facilitator's `/settle`. There is no `/verify` call: under `upfront` the transfer already exists on chain, so there is nothing to pre-check that `/settle` does not check. The facilitator may run in-process.

7. **Facilitator** enforces every rule in “Facilitator Settlement Rules”, reading the interaction and its receipt back from the chain (`moi.InteractionByHash`, `moi.InteractionReceipt`), marks the transfer hash spent, and returns the settlement result.
8. **Resource Server** executes the request and returns the resource, attaching the `PAYMENT-RESPONSE` header.

`PaymentRequirements` for `exact`

Context. The wire shape the seller emits in `PAYMENT-REQUIRED`. Three `extra` keys: `paymentFlow`, which core writes itself once the scheme declares the flow; `areFeesSponsored`, which the facilitator advertises through `getExtra` exactly as Stellar does; and `resource`, which the server half copies in so the buyer can sign over it. No `assetTransferMethod`, because MOI has only one way to move a native asset.

```
{
  "scheme": "exact",
  "network": "moi:devnet",
  "asset": "0x9a8b…",                    // 32-byte asset identifier
(MAS0 or MASN), hex
  "amount": "1000",                       // atomic units, decimal
string
  "payTo": "0x00000000…",                 // seller's 32-byte
participant identifier, hex
  "maxTimeoutSeconds": 60,
  "extra": {
    "paymentFlow": "upfront",             // reserved key, emitted by
core for non-authorization flows
    "areFeesSponsored": false,            // from the facilitator: the
buyer pays its own fuel
    "resource": "https://…"               // the resource URL, so the
buyer can sign over it
  }
}
```

Field definitions:

- `network` — a `moi:*` CAIP-2 identifier from Supported Networks.
- `asset` — the 32-byte identifier of a protocol-native asset (MAS0, or MASN for KMOI). These assets carry no on-chain decimals or symbol, so amounts are always atomic units.
- `amount` — atomic units, decimal string.
- `payTo` — the seller’s participant identifier.
- `maxTimeoutSeconds` — bounds the claim’s `validBefore`.
- `extra.paymentFlow` — always `"upfront"` for this mechanism. This is a protocol-reserved key and core writes it onto the wire itself once the scheme declares the flow; the scheme does not set it by hand.

- `extra.areFeesSponsored` — always `false`. The facilitator advertises it through `getExtra`, as the Stellar scheme does; core merges it into the requirements. The buyer, or a fee payer it names, pays the fuel.
- `extra.resource` — the URL of the resource being purchased, copied from the request's `ResourceInfo`. The buyer signs over it and the facilitator checks it (rule 5).
- `extra.assetTransferMethod` — absent. MOI offers one way to move a native asset, so the scheme declares the SDK's `"default"` asset transfer method, and core strips the key from the wire.

PAYMENT-SIGNATURE Header Payload

> **Context.** The buyer's proof. The claim repeats the quoted fields so the facilitator can compare them, and adds the transfer hash, the resource, a validity window and a nonce. `keyId` MUST be 0 because rule 3 derives the paying account from the public key, and only key 0 derives to it.

The `payload` member of the standard `PaymentPayload` envelope carries:

```
{
  "publicKey": "02a1…",           // buyer's compressed public key,
hex, no 0x prefix
  "keyId": 0,                     // MUST be 0: the key the account
identifier derives from
  "signature": "0x…",             // ECDSA secp256k1 over the
canonical claim bytes
  "claim": {
    "from":        "0x00000000…", // the paying account
    "network":     "moi:devnet",  // MUST equal requirements.network
    "to":          "0x00000000…", // MUST equal requirements.payTo
    "asset":       "0x9a8b…",     // MUST equal requirements.asset
    "value":       "1000",        // atomic units, decimal string
    "txHash":      "0x…",         // interaction hash of the ALREADY-
SETTLED transfer
    "resource":    "https://…",   // what was bought; binds the
payment to one request
    "validAfter":  "1757000000",  // unix seconds
    "validBefore": "1757000060",
    "nonce":       "0x…"          // 32 random bytes; see rule 8
  }
}
```

Canonical claim bytes

> **Context.** The one place this spec defines a convention of its own rather than describing MOI wire behaviour. A positional JSON array with a domain separator, and an exact encoding rule so a Go or Python facilitator produces the same bytes as the TypeScript client. The escaping paragraph was added after review because `JSON.stringify` and Go's default encoder disagree on <, > and &.

Signer and verifier MUST produce identical bytes, so the claim is serialized positionally, with a domain separator as the first element:

```
["moi-x402-payment-claim-v1", from, network, to, asset, value, txHash, resource, validAfter, validBefore, nonce]
```

The UTF-8 encoding of that JSON array is the message signed. The domain separator prevents a signature produced here from being valid as a signature over anything else the same key signs.

Encoding MUST be exact so that implementations in different languages agree: no whitespace between elements; every element a JSON string; string escaping limited to `"`, `\` and control characters U+0000–U+001F (as `\uXXXX`, or the short forms `\n`, `\r`, `\t`, `\b`, `\f`); no escaping of `/`, <, >, &, and no `\u` escaping of any character above U+001F. This is what JavaScript's `JSON.stringify` produces; other encoders MUST be configured to match (for example, Go's `SetEscapeHTML(false)`, Python's `ensure_ascii=False` with `separators=(",", ":")`).

Why a claim exists at all

> **Context.** Answers the obvious reviewer question: why not just send the transfer hash. Because the hash is public the moment it is mined.

The transfer hash is public the moment it is mined; anyone watching the chain can copy it. The claim is what stops a lifted hash being redeemed by someone other than the payer: it is signed by a key that must derive to the paying account, it names one `resource`, it carries a validity window and a nonce, and the transfer hash is marked spent on settlement.

Facilitator Settlement Rules (MUST)

> **Context.** Eight rules. Each is one step in the package's `settle`, and each failure returns a namespaced `errorReason` (the README in PR 3 maps reasons to rules). The rules run in `/settle` because under `upfront` core never calls `/verify`.

Because the flow is `upfront`, these rules run inside `/settle`. A facilitator MUST reject the settlement if any rule fails. `SettleResponse.errorReason` is a free-form string; this spec does not define a vocabulary, and implementations SHOULD namespace their reasons as the Stellar scheme does.

1. Shape

> **Context.** Presence check, before anything is parsed.

`claim`, `signature` and `publicKey` MUST be present.

2. Signature

> **Context.** ECDSA secp256k1 over the canonical bytes, verified through js-moi-sdk's own `Signer.verify` (blake2b-256 digest). Nothing is re-implemented.

The ECDSA secp256k1 signature over the canonical claim bytes MUST verify against `publicKey`.

3. Key controls payer

> **Context.** The rule that defeats a lifted hash: the key that signed must derive to `claim.from`. Derivation is the SDK's `Wallet.getIdentifier` (v0 tag, 24 key bytes, variant 0), so sub-accounts fail today; a later version may derive with the claimed variant.

The participant identifier derived from `publicKey` MUST equal `claim.from`. A valid signature alone could still merely claim someone else's transfer; this rule defeats a lifted hash.

Because the identifier derives from one key, the claim MUST be signed with that key (`keyId` `0`). Accepting the account's other keys would require checking `publicKey` against the account's registered keys on chain instead of deriving; a later version MAY add that. The derivation uses variant `0`, so a sub-account identifier (non-zero variant) fails this rule; a later version MAY derive with the variant the claim names.

4. Claim matches quote

> **Context.** `network` was added to this rule on 10 September, with `network` added to the claim, so a claim signed for one MOI network cannot settle on another (identifiers are the same on every network).

`claim.network` MUST equal `requirements.network`; `claim.asset` MUST equal `requirements.asset`; `claim.to` MUST equal `requirements.payTo`; `claim.value` MUST be at least `requirements.amount`.

5. Resource binding

> **Context.** The check that was dead in the earlier draft. The resource reaches both halves by different routes and the facilitator must compare them, failing closed if core's `paymentPayload.resource` is absent.

`claim.resource` MUST equal the resource being purchased. Without this check the `resource` field is signed but unenforced, and a claim made for one resource would settle a request for another on the same seller.

The resource reaches both halves by different routes, and the facilitator MUST compare them: the seller's scheme copies it into `requirements.extra.resource` (its `enrichPaymentRequiredResponse` hook receives the `ResourceInfo`), the buyer signs that value into the claim, and core carries the authoritative `ResourceInfo` on `PaymentPayload.resource`. The facilitator MUST compare `claim.resource` against `paymentPayload.resource.url`, and MUST fail closed rather than skip the comparison if that field is absent.

6. Freshness

> **Context.** Two checks: now inside the window, and the window no wider than the seller's `maxTimeoutSeconds`. The earlier draft let the buyer widen its own window.

The current time MUST lie within `[validAfter, validBefore]`, and `validBefore - validAfter` MUST NOT exceed `requirements.maxTimeoutSeconds`. A buyer that widens its own validity window beyond the quoted timeout MUST be rejected.

7. The transfer settled on chain

> **Context.** The rule that makes this `upfront`: the facilitator reads the transfer back and trusts nothing in the payload. The JSON is a real devnet transfer, verified live. The receipt carries neither beneficiary nor amount, so both reads are required and the calldata is POLO-decoded. `ASSET_INVOKE` is op type 5 and the callsite is `Transfer` (go-moi constants, verified). A refused transfer still has a hash and a receipt, hence the status check.

The facilitator MUST read the transfer back from the chain and MUST NOT trust the payload. The receipt carries neither the beneficiary nor the amount, so the read-back takes both objects: `moi.InteractionByHash(claim.txHash)` for the submitted operation and `moi.InteractionReceipt(claim.txHash)` for its result.

A settled transfer on Voyage devnet returns:

```
// moi.InteractionByHash
{
  "sender": {
    "id": "0x00000000bfa45bc0…",
    "sequence_id": "0x4f",
    "key_id": "0x0"
  },
  "ix_operations": [
    {
      "type": 5,                                    // ASSET_INVOKE
      "payload": {
        "asset_id": "0x108000004cd973c4…",
        "callsite": "Transfer",
        "calldata": "0x0d6f0665…"                   // POLO document
      }
    }
  ]
}

// moi.InteractionReceipt
{
  "status": 0,                                      // interaction-
level: 0 is success
  "fuel_used": "0x12b",
```

```
  "from": "0x00000000bfa45bc0…",
  "ix_operations": [
    { "tx_type": "0x5", "status": 0, "data": { "outputs": "0x0d0f",
"error": "0x" } }
  ]
}
```

The facilitator MUST reject the settlement unless all of the following hold:

- The interaction and its receipt both resolve for `claim.txHash`.
- `receipt.status` is `0`. A refused transfer still yields an interaction hash and a mined receipt, so possession of a hash proves nothing by itself.
- `interaction.sender.id` equals `claim.from`.
- The interaction contains an operation of `type` 5 (`ASSET_INVOKE`) whose payload's `asset_id` equals `requirements.asset` and whose `callsite` is `"Transfer"`. MOI has no dedicated transfer opcode; a transfer is an asset call routed by callsite name.
- That operation's `calldata`, POLO-decoded, yields a document with a `beneficiary` equal to `claim.to` and an `amount` of at least `claim.value`.

8. Replay

> **Context.** The transfer hash, not the nonce, is the single-use key. The spent-store must be durable and its insert atomic; the nonce only makes two claims over one transfer distinguishable.

`claim.txHash` MUST NOT have been redeemed before. On success the facilitator MUST record it in a spent-store before returning. The spent-store is the replay boundary and MUST be durable across process restarts in production.

The transfer hash, not the `nonce`, is what makes a payment single-use: one settled transfer can be redeemed once. The `nonce` exists so that two claims over the same transfer are distinguishable as messages, and facilitators MAY ignore it.

PAYMENT-RESPONSE Header Payload

> **Context.** `transaction` is the buyer's own hash, confirmed rather than executed, which is the visible difference from an `authorization` mechanism.

```
{
  "success": true,
  "transaction": "0x…",       // claim.txHash — the buyer's own
transfer, confirmed not executed
  "network": "moi:devnet",
  "payer": "0x00000000…",
  "amount": "1000"
}
```

On failure, `success` is `false`, `errorReason` names the rule that failed, and `transaction` carries the submitted hash when one was supplied.

Duplicate Settlement Mitigation (RECOMMENDED)

> **Context.** The same section the SVM and NEAR specs carry. On MOI the race is between the chain read and the spent-store write, so the mitigation is an atomic insert-if-absent shared across replicas.

Vulnerability

If the same payload reaches `/settle` concurrently, both executions can pass rule 7 (the transfer is genuinely on chain) before either records the hash, and one payment buys twice. This is the same race the SVM and NEAR schemes document; on MOI the window is the gap between the chain read and the spent-store write.

Recommended Mitigation

The spent-store write MUST be atomic check-and-set (insert-if-absent) rather than separate has/add steps, and SHOULD be shared across facilitator replicas. Sellers SHOULD treat the `resource` binding as a second key: one `(txHash)` marks the payment spent, and the claim's `resource` confines what it could ever have bought.

Security Considerations

> **Context.** Four items. Overpayment is accepted (rules compare with >=); clock skew narrows the window; identifiers are derived from keys, which is why the network is inside the signed claim and why a facilitator must confirm which network it is reading; the buyer's fuel never becomes the seller's problem.

Overpayment is accepted. Rules 4 and 7 compare with >=, so a buyer that sends more than the quoted amount still settles. Underpayment is rejected. Sellers wanting exact-amount semantics MUST compare for equality instead.

Clock skew narrows the window. `validAfter` and `validBefore` are the buyer's own timestamps. Skew between buyer and facilitator shortens the usable window and can reject an otherwise valid claim; sellers SHOULD quote `maxTimeoutSeconds` with that margin in mind. A later version MAY anchor freshness to the transfer's tesseract timestamp instead of the buyer's clock.

Identifiers are derived, not allocated. A participant identifier contains a 24-byte slice of the account's compressed public key. Rule 3 relies on that derivation: producing a key that derives to a chosen account means grinding keypairs against 24 fixed bytes, which is computationally infeasible. It also means the same identifier exists on every MOI network, so a facilitator MUST confirm it is reading the network named in `requirements.network` and MUST NOT infer the network from an endpoint URL. The claim carries `network` in its signed bytes for the same reason: the buyer commits to the network, and rule 4 checks it.

The buyer's fuel is not the seller's concern. A buyer with insufficient fuel fails before settlement and never produces a claim; the seller sees no request.

Appendix

> **Context.** Describes only wire-visible behaviour because the node and VM repositories are private. Every claim here was checked against go-moi source and the live devnet (see PR 3 `STATUS.md`): the identifier layout, the two field names that differ between the signing schema and the RPC response, and `fee_payer` versus the pre-0.13.0 `payer`.

MOI in brief

MOI is an account-centric protocol. Each participant account holds its own state and its own ordered sequence of tesseracts; there is no single global chain of blocks. Fungible assets are protocol-native (the MAS0 standard, and MASN for the KMOI token) rather than contract-deployed, and interactions — the signed unit of state change — are signed by the sending account with ECDSA over secp256k1. Because the buyer can submit its own transfer directly, this scheme settles first and proves second.

Interaction and receipt anatomy

As observable through the JSON-RPC surface and the reference client (js-moi-sdk). The node and VM implementations are not public, so this spec describes only wire-visible behavior.

An interaction envelope contains: `sender { id, sequence, key_id }`, an optional `fee_payer` (named `payer` before moipod v0.13.0; when set, that account co-signs and pays the fuel), `fuel_price` and `fuel_limit`, a `funds` list, `ix_operations` (each `{ type, payload }`), a `participants` list (account id, lock type and a notary flag — a scheduling and co-signing declaration, not a permission), and consensus preferences. The whole envelope is POLO-encoded and signed with ECDSA over secp256k1; `sender.key_id` selects which of the account's keys signed.

A participant identifier is 32 bytes: a tag byte, a flags byte, two standard bytes (zero for a participant), a 24-byte fingerprint sliced directly from the account's compressed public key (the 24 bytes after its parity byte), and a 4-byte variant carrying the sub-account index. It derives from the key, not from a network, so the same identifier can exist on every MOI network — one reason explicit network identification matters for this scheme.

A receipt contains `ix_hash`, an interaction-level `status`, `fuel_used`, `from`, and per-operation results (`{ tx_type, status, data { outputs, error } }`). Interaction-level status `0` is success; `tx_type` comes back as a hex string rather than a number.

Two names differ between the signing schema and the RPC response for the same field: the envelope is signed with `sender.sequence` and a `participants` list, and `moi.InteractionByHash` returns them as `sender.sequence_id` and `ix_participants`. Implementations reading settled interactions MUST use the RPC spelling.

Native assets carry no on-chain decimals or symbol; all amounts in this scheme are atomic units, and no `"$0.10"`-style pricing is defined for MOI.

POLO

POLO (Prefix Ordered Lookup Offsets) is MOI's deterministic serialization format, specified at github.com/sarvalabs/polo with public Go and JavaScript implementations. Both the interaction envelope and asset calldata are POLO-encoded.

References

- MOI protocol: https://moi.technology · docs: https://docs.moi.technology
- js-moi-sdk (reference client): https://www.npmjs.com/package/js-moi-sdk
- POLO serialization: https://github.com/sarvalabs/polo (js-polo, go-polo)
- MOI CAIP-2 namespace (in progress): ChainAgnostic/namespaces `moi/`
- x402 v2 specification, section 6.1 (payment flow models)